דף סיכום בחינה

מזהה סטודנט:

מזהה קורס: 20594   שם קורס: מערכות הפעלה

מספר מבחן: 0001

מספר שאלה בטופס בחינה זה	ניקוד מירבי	ציון	התשובה שסומנה	התשובה הנכונה	מיקום השאלה בטופס המקור	מיקום התשובה שסומנה בטופס המקור
1	25.00	25.00	-	-	1	-
2	19.00	18.00	-	-	2	-
3	11.00	10.00	-	-	3	-
4	5.00	4.00	-	-	4	-
5	5.00	5.00	-	-	5	-
6	5.00	3.00	-	-	6	-
7	5.00	5.00	4	4	7	4
8	5.00	5.00	1	1	8	1
9	5.00	5.00	4	4	9	4
10	5.00	5.00	2	2	10	2
11	5.00	5.00	4	4	11	4
12	5.00	5.00	2	2	12	2

כמות תשובות נכונות (עבור שאלות סגורות) : 6

ציון בחינה סופי : 95.00

מקרא צבעים

תשובה שגויה
שאלה שלא נענתה
שאלה מבוטלת

הבחינה הבדוקה בעמודים הבאים

האוניברסיטה
הפתוחה

דף תשובות

תאריך בחינה	מספר הקורס	מספר תעודת זהות	מספר גרסה
270623	20594	[redacted]	0001

0	●	●	●	○
1	○	○	○	●
2	○	○	○	○
3	○	○	○	○
4	○	○	○	○
5	○	○	○	○
6	○	○	○	○
7	○	○	○	○
8	○	○	○	○
9	○	○	○	○

הוראות למילוי דף התשובות

1. נא למלא תשובה אחת בלבד על ידי השחרת העיגול - ●
2. את ההשחרה יש לבצע בעט שחור או כחול בלבד!
3. השימוש בטיפקס או מרקרים אסור!
4. כדי לתקן סימן, יש לסמן X על התשובה הלא נכונה - ⊗
   ולהשחיר את התשובה הנכונה - ●
5. כדי לחזור לסימון המקורי, יש לסמן X על התשובה הלא רצויה,
   ולהשחיר מחדש את סימן ה- ⊗ באופן שיכסה המחיקה - ●

שאלה/תשובה	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	שאלה/תשובה
א	○	○	○	○	○	○	○	●	○	○	○	○	○	○	○	○	○	○	○	○	א
ב	○	○	○	○	○	○	○	○	○	●	○	●	○	○	○	○	○	○	○	○	ב
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ג
ד	○	○	○	○	○	○	●	○	●	○	●	○	○	○	○	○	○	○	○	○	ד
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ה
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ו

שאלה/תשובה	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	שאלה/תשובה
א	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	א
ב	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ב
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ג
ד	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ד
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ה
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ו

שאלה/תשובה	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	שאלה/תשובה
א	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	א
ב	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ב
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ג
ד	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ד
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ה
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ו

517 שאלון ג 82.53.5 M1

ש מ

ת.ז

מספר סידורי: 14

מספר תעודת זהות
יש לרשום את כל תשע הספרות

האוניברסיטה הפתוחה

ח' בתמוז תשפ"ג

27

ביוני 2023

מס' שאלון - 517

סמסטר 2023ב

20594 / 4

מס' מועד 82

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **12** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלקים א ו - ב מופיעות שאלות פתוחות. ענו תשובות מלאות,
בכתב קריא ובקיצור נמרץ. אין חובה להשתמש בכל השורות המוקצות
לצורך התשובות, אך אין לחרוג מהמקום המוקצה.
את יש לכתוב בשאלון הבחינה.

ג. בחלק ג (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה
הנכונה ביותר מבין התשובות המוצעות.
את התשובות בחלק ג' עליכם לסמן בטופס המחשב המצורף שבסוף השאלון.

חומר עזר:
כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.
אין להכניס חומר דיגיטלי!

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

517 שאלון 82.53.5 M1

הנחיות למבחן:

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. בחלקים א' ו ב' מופיעות שאלות פתוחות. ענו **תשובות מלאות, בכתב קריא** ו**בקיצור נמרץ**. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.
2. בחלק ג' (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה הנכונה ביותר מבין התשובות. **את התשובות בחלק ג' עליכם לסמן בטופס המחשב המצורף שבסוף השאלון.**
3. כל חומר עזר אסור לשימוש בזמן הבחינה פרט למחשבון פשוט
4. משך הבחינה - שלוש שעות.

בהצלחה!

חלק א (55 נקודות)

ענו על שאלות 1-3.

שאלה 1 (25 נקודות)

נתונה מערכת מחשב שבה המעבד CPU תומך בפקודה אטומית COMPARE AND – CAS .SWAP

להלן פסודוקוד המתאר את פעולת הפקודה :

```
int compare_and_swap(int *value, int expected, int new_value) {
int temp = *value;
 if (*value == expected) *value = new_value;
 return temp; }
```

← הערך הישן

א. (7 נק')
על בסיס הפקודה הזאת ממשו את פרוטוקול **פשוט ביותר בסיסי** לכניסה ויציאה של קטע קריטי המיועד לסנכרון תהליכונים THREADS הרצים במקביל.
הפרוטוקול חייב להבטיח את **מניעה הדדית והתקדמות**, אך **אינו צריך להבטיח המתנה מוגבלת**(העדר הרעבה). הסבירו כיצד הפרוטוקול עובד.

תשובה והסבר :

נשתמש ב mutex שמאותחל ל-0.

```
while (true) {
while (compare_and_swap(&mutex, 0, 1))
        /* noop */
/* critical section */
```

~~[illegible]~~,

mutex = 0; // open lock

}

מנגנון פשוט, כמו TSL במשהו. בהתחלה, מנסים לנעול את המנעול. אם הוא כבר נעול אז יוחזר 1 ונמשיך בלולאה, אם הוא פתוח אז הוא ינעל ויוחזר 0 ונכנס לקטע הקריטי. לכן הפרוטוקול מבטיח מניעה הדדית, אין ~~[illegible]~~ יתכן הרעבה כי ייתכן ולא יודעים איזה תהליך ייכנס פעם הבאה לקטע.

ב. (12 נקי׳)

הוצע הפרוטוקול הבא(**לא הכי פשוט**) לפתרון בעיית קטע קריטי המבוסס על אותה פקודה:

```
while (true) {
waiting[i] = true;
key = 1;
while (waiting[i] && key == 1)
key = compare and swap(&lock,0,1);
waiting[i] = false;
/* critical section */
j = (i + 1) % n;
while ((j != i) && !waiting[j])
j = (j + 1) % n;
if (j == i)
lock = 0;
else
waiting[j] = false;
/* remainder section */
}
```

// כל עוד זה לא ~~[illegible]~~ ממתינים

אם lock==0, הוא ישנה ל-1, ומחזיר 0. אם הוא 1 אז הוא נשאר 1.

// j<n, j++ לדוגמא:

// מחפשים תהליך שרוצה

// j<n, j++

// [illegible]

האם הפרוטוקול המוצע עומד בכל דרישות פתרון קטע קריטי כולל המתנה מוגבלת?

הוכיחו שכן או הפריכו ע״י דוגמא נגדית.

תשובה והסבר:

כן. מניעה הדדית: ~~[illegible]~~ נניח בשלילה שיתכן שיהיו 2 תהליכים לפחות בקטע הקריטי ביחד. נראה שהמשתנה lock יכול להכניס רק תהליך אחד לקטע הקריטי (בגלל אופן הפעולה של CAS) או רק אם waiting[i]=false שהוא לא יכול לשנות. אבל זה קורה רק אחרי הקטע הקריטי (כלומר ההשמה הזו יכולה להתרחש רק אחרי שתהליך אחר סיים קטע קריטי). סתירה להנחה. ~~[illegible]~~ התקדמות: אם אף תהליך לא רוצה להכנס, אז lock=0 והקטע הקריטי פנוי. ואז אף תהליך לא יחכה לשווא לתהליך אחר. כמו כן CAS אטומית.

המתנה מוגבלת: לאחר שi סיים קטע קריטי, הוא עובר על כל התהליכים האחרים ובודק האם אחד מהם מחכה להכנס ותקוע בלולאה בשורה 4. אם כן הוא משחרר אותו. הבדיקה היא לפי סדר ולכן לא יתכן שתהליך כלשהו יורעב. וכמו כן, אין תהליך בתוך הקטע קריטי שלא מונע מתהליך אחר להכנס.

נא לא לכתוב בשוליים

ג. (6 נק׳)

האם הפרוטוקול המוצע מבטיח גם הוגנות(מי שניסה להיכנס ראשון, ייכנס ראשון אחרי שקטע קריטי יתפנה)? הוכיחו שכן או הפריכו ע״י דוגמא נגדית.

25
(1)

תשובה והסבר:

הכוונה לאין בוסים.

לא

: כפי שנאמר, הבעיה היא לפי הסבר, נניח שתהליך 1 הגיע.
lock=0 ולכן הוא נכנס, מסיים קטע קריטי. נניח שבזמן הזה
הגיע תהליך 3, נתקע בלולאה ואחריו הגיע תהליך 2 וגם נתקע.
תהליך 1 יבצע את ההשמה j=2, ואז בלולאה נגלה ש waiting[2]=true
ולכן נצא ממנה.. j!=i ולכן, נבצע waiting[2]=false
ותהליך 2 ייכנס לקטע הקריטי. לפני תהליך 3, אפילו שתהליך
3 הגיע קודם.

המשך הבחינה בעמוד הבא

שאלה 2 (19 נקודות)

נתונה מערכת מחשב שתומכת בדפים בגדלים שונים : 4 Kbyte ו 4 Mbyte בלבד. כל דף בכל גודל יכול להימצא בכל מקום בזיכרון הפיזי RAM. תהליך שמבקש הקצאת זיכרון ראשי 8 Mbyte או יותר מקבל אותו בדפים בגודל 4 Mbyte ובהקצאות קטנות יותר בדפים בגודל 4 Kbyte. המרחב הווירטואלי מחולק לשני אזורים שווים, בחלק הנמוך גודל הדף 4 Kbyte ובחלק הגבוה 4 Mbyte **במרחב הזיכרון הפיזי אין חלוקה כזאת.**

הקומפיילר המיוחד שפותח בשביל המערכת ופונקציות הספרייה של שפת תכנות קובעים כתובות וירטואליות של משתנים בהתאם(בכתובות גבוהות או נמוכות, לא משנה מה הדרך לעשות את זה).

א. (9 נק')

תארו את האלגוריתם(את הדרך) של MMU לתרגום הכתובת הוירטואלית לכתובת פיזית. ציינו האם נדרשים שינויים בצורת הכתובת הוירטואלית ובטבלת הדפים(בהשוואה לצורה רגילה) כדי לאפשר את תרגום הכתובת ואם כן, מה הם. הדגישו במיוחד את שיטת חישוב מספר מסגרת זיכרון פיזית ואת הגודל שלה.

MMU קודם מגלה האם הכתובת הוירטואלית שייכת לכתובות הגבוהות או הנמוכות (למשל ע"י הסיבית הכי שמאלית בכתובת). אם הכתובת שייכת לחצי הנמוך, אז גודל דף הוא 4KB כלומר 2^{12} bytes, כלומר צריך 12 סיביות ~~[illegible]~~ להיסט, ואם לחצי הגבוה אז גודל דף 4MB = 2^{22} bytes וצריך 22 סיביות להיסט. ה MMU לוקח את שאר הסיביות שמייצגות את מס' הדף הוירטואלי, פונה לטבלת הדפים, ומוצא מס' מסגרת מתאים. שימו, כרגיל בזיכרון הפיזי אין חלוקה כמו שצוין, גודל כל מסגרת לפי גודל הדף הנמוך - 4KB, ועבור דפים בגודל 4MB נצטרך להקצות 2^{10} מסגרות. כמו ובטבלת הדפים תצטרך לעדכן שינוי: לסמן להקצאה של 2^{10} מסגרות לכתובות וירטואליות בחצי הגבוה.

ב. (5 נק')

מה החיסרון העיקרי, חוץ מקיטוע פנימי, ויתרון העיקרי בשימוש בדפים בגודל משתנה בצורה המתוארת בשאלה?

תשובה והסבר :

יתרון: נוכל לבחור את גודל הקובץ בהתאם לצורך, ולכן נוכל לחסוך בזיכרון ראשי, ולהקטין ריסוק פנימי.

חסרון: תקורה גבוהה בעת הקצאה כי נצטרך ~~[illegible]~~ מסגרות מהזיכרון הפיזי, במקרה של גודל 4MB.

נא לא לכתוב בשוליים

18
(2)

ג. (5 נק׳)

הציעו פתרון לחסרון שצוין בסעיף ב׳ .

הוא לא מחולק!

תשובה והסבר :

זה שינוי השיטה ולא פתרון לחסרון

אפשר למשל לחלק את הזיכרון הפנוי על כן לשני חצאים

עם מסגרות בגודל שונה , 4KB ו4MB , ובכך

נבצע התאמה של גודל דפים לגודל מסגרות , ולא

נצטרך להביא המון בלוקים עבור דפים בגודל 4MB.

שאלה 3 (11 נקודות)

א. (5 נק׳)

הסבירו בקצרה את המטרה העיקרית ואת אופן פעולתו של רכיב ה- DMA במערכת מחשב מודרנית.

תשובה והסבר :

רכיב ה DMA (Direct Memory Access) מאפשר לבקרי ה I/O לגשת למקום בזיכרון הראשי שהוקצה עבור קלט - פלט באופן ישיר, במקום לעבור דרך ה CPU. והדבר מאפשר ל CPU להיות פנוי יותר ולבצע את הפקודות במערכת.

ב. (6 נק׳)

מה הן החסרונות של שימוש ב רכיב ה-DMA ? האם הוא יכול לגרום להאטת ביצוע פקודות חישוביות?

מה הן הדרכים להתמודדות עם החסרונות(ציינו את הקיימות או הציעו חדשות)?

10

(3)

תשובה והסבר :

נעילת דפים

רכיב ה DMA יכול להוביל ל"צוואר בקבוק" בעקבות פעולות קלט / פלט רבות, למשל אם נבחר גודל יחידת מען גדול מ.ב. אז יקח יותר זמן להעביר אותה, וזה יחסום את האפיק, אם נבחר גודל יחידת מען קטן מ.ב. אז אומנם יעבור יותר מהר אבל יגדיל את התקורה בעקבות פעולות רבות מ.ב.".

הדרכים להתמודד: כמובן וה - cpu עסוק פחות זה DMA ניתן לאפשר לו לעשות חלק מהפעולות. ואז כן ניתן לבחור גודל יחידת מען כך שלא יהיה גדול מ.ב. ולא קטן מ.ב.

המשך הבחינה בעמוד הבא

חלק ב (15 נקודות)

ענו על שלוש השאלות 4-6.

5 נקודות לשאלה.

שאלה 4

למה יש מספר לכל קריאת מערכת SYSTEM CALL ? האם ניתן לממש את קריאת מערכת בלי שיהיה לה מספר?

תשובה והסבר :

4
(4)

לכל קריאת מערכת יש מספר כיוון שקריאות המערכת ממוספרות במחשב, וכך המספר שמועבר אליה מערכת ההפעלה יכולה לזהות למערך הזה ולבחור את קריאת המערכת הנכונה ע"י המספר. בלי מספר, מערכת ההפעלה לא תדע איזו קריאה להפעיל. ולכן, [illegible] לממש קריאת מערכת ללא מספר.

למה דווקא מספר?

בגלל וקטור הפסיקות

שאלה 5

צוות מתכנתים עובד על הוספת תמיכה בקונטיינרים למערכת דמוית LINUX שלא תמכה בהם לפני זה.

אחרי הוספת המנגנות יצירת הקונטיינרים(קריאות מערכת חדשות, כמו UNSHARE למשל) והגדרת המשאבים שלהם, האם יהיה צורך לשנות את קוד מקור של **קריאות מערכת** קיימות?

אם עניתם לא, תסבירו איך קריאות המערכת הקיימות ימשיכו לעבוד בצורה תקינה.

אם עניתם כן, תסבירו בצורה כללית ותנו דוגמא אלו קריאות מערכת יהיה צורך לשנות ומה השינוי העיקרי הנדרש באופן ביצוען.

הערה : התשובה הכללית מדי בסגנון "צריך להתחשב בקונטיינרים" לא תתקבל כי לא מהווה הסבר.

5
(5)

תשובה והסבר :

כן. למשל כמו שעשינו בממ"ן 12, שינינו את הקריאה ps שתציג רק את התהליכים הנוכחיים, את pid שלהם ו- extpid. לאחר הוספת התמיכה בקונטיינרים, לכל תהליך יש pid מקומי ו- pid גלובלי. שינויים אלה לא היו נחוצים אילו אנחנו לא בתוך קונטיינר. גם בממ"ן 13 שינינו את קריאת המערכת mount כך שתחפש מקום פנוי עבור קונטיינר חדש (לפני זה היא החזירה את אותו מספר לכולם).

82.53.5 M1 שאלון 517

שאלה 6

במערכת XV6 שתומכת בקונטיינרים(אבל לא בקונטיינרים מקוננים (nested הפעיל רצף של הפקודות אחרי הרצה נקייה של המערכת:

פקודות

```
$ pouch start c1
Pouch: c1 starting
$ pouch connect c1

tty0 connected
$ mkdir mntdir
$ mount mntdir -t cgroup
cgroup filesystem already mounted
$
```

משותף לכל הקונטיינרים

?

א. למה פקודת mount לא הצליחה עם ההודעה שהיא "already mounted", הרי מעולם המשתמש לא הפעיל ישירות את הפקודה הזאת? על ידי מי ולמה(בשביל איזו מטרה) פקודת mount כבר הופעלה?

ב. למה בעיה זהה לא תתרחש מתי שתבוצע פקודה שיוצרת קונטיינר נוסף?

3
(6)

תשובה והסבר:

✓ א. פקודת mount הופעלה כחלק מהגדרה שיצרה את הקונטיינר. כשיוצרים קונטיינר חדש רוצים ליצור לו מערכת קבצים משלו, שתהיה מבודדת ממערכת הקבצים הקודמת.

✗ ב. כשיוצרים קונטיינר ~~חדש, הפקודה mount מופעלת אוטומטית.~~ אבל כן עוד לא התחברנו אליו (כלומר לא עשינו connect) אז אנחנו מחוץ לקונטיינר ואין בעיה לעשות mount כי mount הופעלה כבר בתוך הקונטיינר ולא מחוץ לו.

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

חלק ג (30 נקודות)

ענו על שש שאלות רב-ברירה (אמריקאיות). משקל כל שאלה 5 נקודות.

בכל שאלה יש לבחור את התשובה הנכונה **ביותר** ולהקיף בעיגול את אות התשובה שבחרתם.

שאלה 7

מערכת הקבצים של מערכת הפעלה מסוימת עושה שימוש ברשימות גישה (access control lists – ACL). נניח כי במערכת מוגדרים 3 משתמשים **U1** , **U2** , **U3** . משמעות ההרשאות : RWX היא הרשאות גישה **R**ead, **W**irite, **E**execute . להלן רשימת ACL של הקבצים במערכת:

F 1	**U1: rw , U2: rwx**
F 2	**U1: w, U3: rw**
F 3	**U1: rx, U3: rw**

האם הנתונים מתוך קובץ F2 יכולים להגיע(בדרך ישירה או עקיפה) לידי משתמשים U1 ו U2 ?

א. לא, לאף אחד מהם.

ב. כן, רק ל U1 .

ג. כן, רק ל U2 .

[✓ ד.] כן, גם ל U1 וגם U2 .

שאלה 8

מה נכון לגבי HYPERVISOR מסוג 1 בתצורת bare metal?

[✓ א.] כל משאבי החומרה hardware resources נשלטים **באופן ישיר** ע"י hypervisor .

ב. רק הקצאות זיכרון ראשי ופעולות קלט-פלט עוברות דרך hypervisor , אבל הקצאת זמן המעבד CPU הפיזי מבוצעת **ישירות** ע"י ה OS האורחת **בלי שיתוף** של hypervisor .

ג. רק הקצאות זיכרון ראשי עוברות דרך hypervisor , אבל פעולות קלט-פלט מבוצעות **ישירות** ע"י ה OS האורחת **בלי שיתוף** של hypervisor .

ד. כאשר יישום המשתמש מבצע את system calls ל-OS שרצה על VMM , אותה ה OS **לא תנסה** לבצע את ה- system calls ב kernel mode .

המשך הבחינה בעמוד הבא

שאלה 9

בחרו מידע אשר **איננו מצוי** ב PCB (process control block) של תהליך :

א. מצב של תהליך (ריצה, המתנה וכו׳).

ב. מצביע לטבלת קבצים פתוחים אשר נפתחו ע״י התהליך.

ג. מצביע למפת הזיכרון(טבלת הדפים) של התהליך.

[✓ ד.] מונה מספר בלוקים שנקראו מדיסק ע״י התהליך.

שאלה 10

מהו החיסרון המובהק של שיטת ה- rendezvous(שיטת המפגש) לעומת ה- mailbox ב- message passing?

א. שיטת ה-rendezvous קשה יותר למימוש במערכת הפעלה.

[✓ ב.] שיטת ה-rendezvous פחות גמישה מכיוון שהתהליכים (השולח והנמען) הופכים להיות תלויים זה בזה.

ג. בשיטת ה-rendezvous לא ניתן לממש מניעה הדדית בגישה לקטעים קריטיים (critical sections).

ד. שיטת ה-rendezvous ניתנת למימוש רק אם מעבד CPU הוא בעל ליבה CORE אחת.

שאלה 11

מה מבין המפורטות היא הדרך הנכונה **והסבירה** למניעת deadlock במערכת עם 4 RESOURSES ?

א. כל process ישחרר את כל RESOURCES שהוא **החזיק** בהם(כולל אמצעי סנכרון) לפני שיסתיים. לא מונע [illegible]

ב. כל process יוכל לבקש לא יותר מ 2 RESOURCES . X

ג. כל ה processes יוכלו לבקש ולהחזיר רק RESOURSE אחד בכל נקודת זמן של חיי התהליך. לא סביר בכלל

[✓ ד.] כל ה processes יוכלו לקבל את RESOURCES רק לפי סדר עולה ולשחרר לפי סדר יורד.

המשך הבחינה בעמוד הבא

שאלה 12

מערכת הקבצים של מערכת הפעלה מסוימת משתמשת בשיטת ה I-node עם פרמטרים הבאים :

- גודל הבלוק במערכת הקבצים הוא 2 Kbytes = 2^{11} bytes
- כתובת הבלוק היא 4 בתים (bytes) 2^2
- 10 שדות של ה I-node יכולים להחזיק ישירות כתובת הבלוק בדיסק
- שדה נוסף אחד נועד להחזיק כתובת של ה single indirect block
- עוד שדה נוסף אחד נועד להחזיק כתובת של ה double indirect block
- ועוד שדה נוסף אחד נועד להחזיק כתובת של ה triple indirect block

חשבו מה הגודל המרבי של קובץ שניתן לאכסן ב 1514 בלוקים בסה״כ(כולל נתונים ומצביעים, אבל לא כולל את ה I-NODE עצמו) .

הערה : לצורך החישוב 1 Kbyte = 1024 Byte , 1 MByte = 1024 Kbyte

א. 3028 Kbytes

[✓ ב.] 3020 Kbytes

ג. 3000 Kbytes

ד. 3008 Kbytes

בהצלחה!

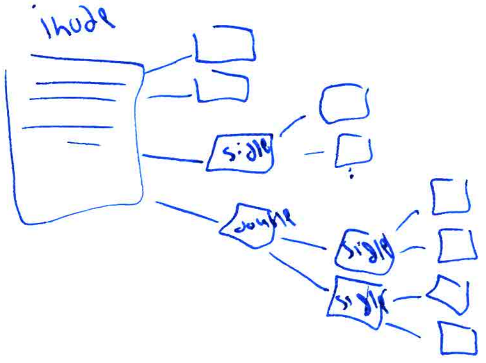
$2^{11}/2^2 = 2^9$

בסינגל ← 2^9 בלוקים

בדאבל 512 בלוקים ← סה״כ 513

בלוק דאבל ← ~~[illegible]~~ ← ~~[illegible]~~

ועוד ~~[illegible]~~ 2 בלוקים ל ~~[illegible]~~ 988 (כי זה מה שאפשרי.



1510 בלוקים של נתונים:

$1510 \cdot 2^{11} = 3020$ KB